Fundamentals of Computer Design

(Computer Architecture: Chapter 1)

Yanyan Shen

Department of Computer Science and Engineering

Agenda

- Introduction
- Classes of Computers
- Defining Computer Architecture
- Trends in Technology
- Trends in Power and Energy in ICs
- 1.6 Trends in Cost
- Dependability
- Measuring Performance
- Quantitative Principles

Impact of Time, Volume, Commoditization

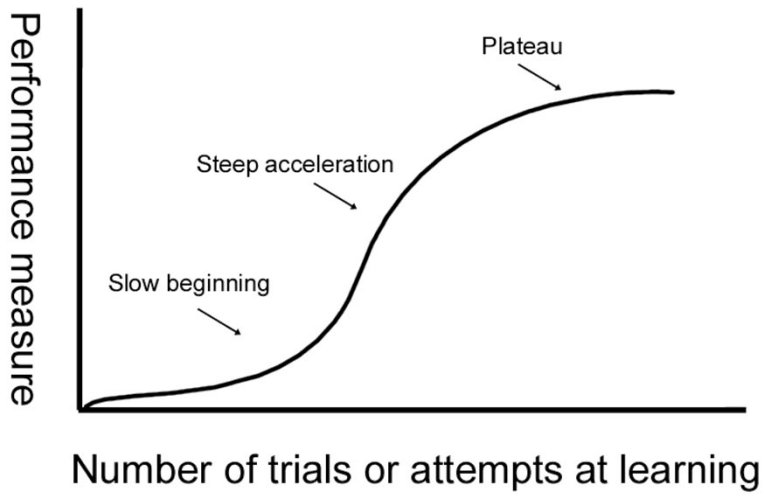
市场规律：随着时间，其成本也会下降

- ***Time***: The cost of a computer component decreases over time, why?
  - The learning curve!
  - ***Yield***: the percentage of manufactured devices that survives the testing procedure

随着技术成熟，成品率会慢慢上升，cost就会下降

- ***Volume*** is another factor in determining cost, why?
  - Amortized cost per computer
  - Microprocessors: price depends on volume, 10% less for each doubling of volume

前期投入更高，后期规模化会慢慢下降



learning curve

规模也是成本来源。平均在每个计算机上的成本，量越大成本就越低。

Impact of Time, Volume, Commoditization

商业化

Commodities are products that are sold by multiple vendors in large volumes and are identical

- Commoditization <u>means that the market is highly competitive</u>
  - 1) The gap between cost and price is **narrowed**
  - 2) It helps clearly define a product, and it increases the **competition** among the suppliers

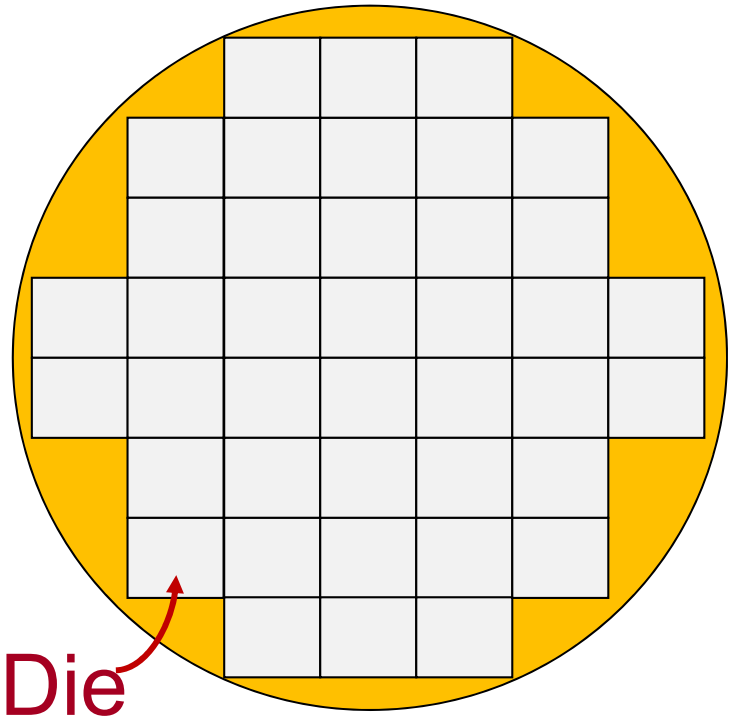
    竞争的情况下，成本和价格之间的gap就会压缩。

Costs of Integrated Circuit (ICs)

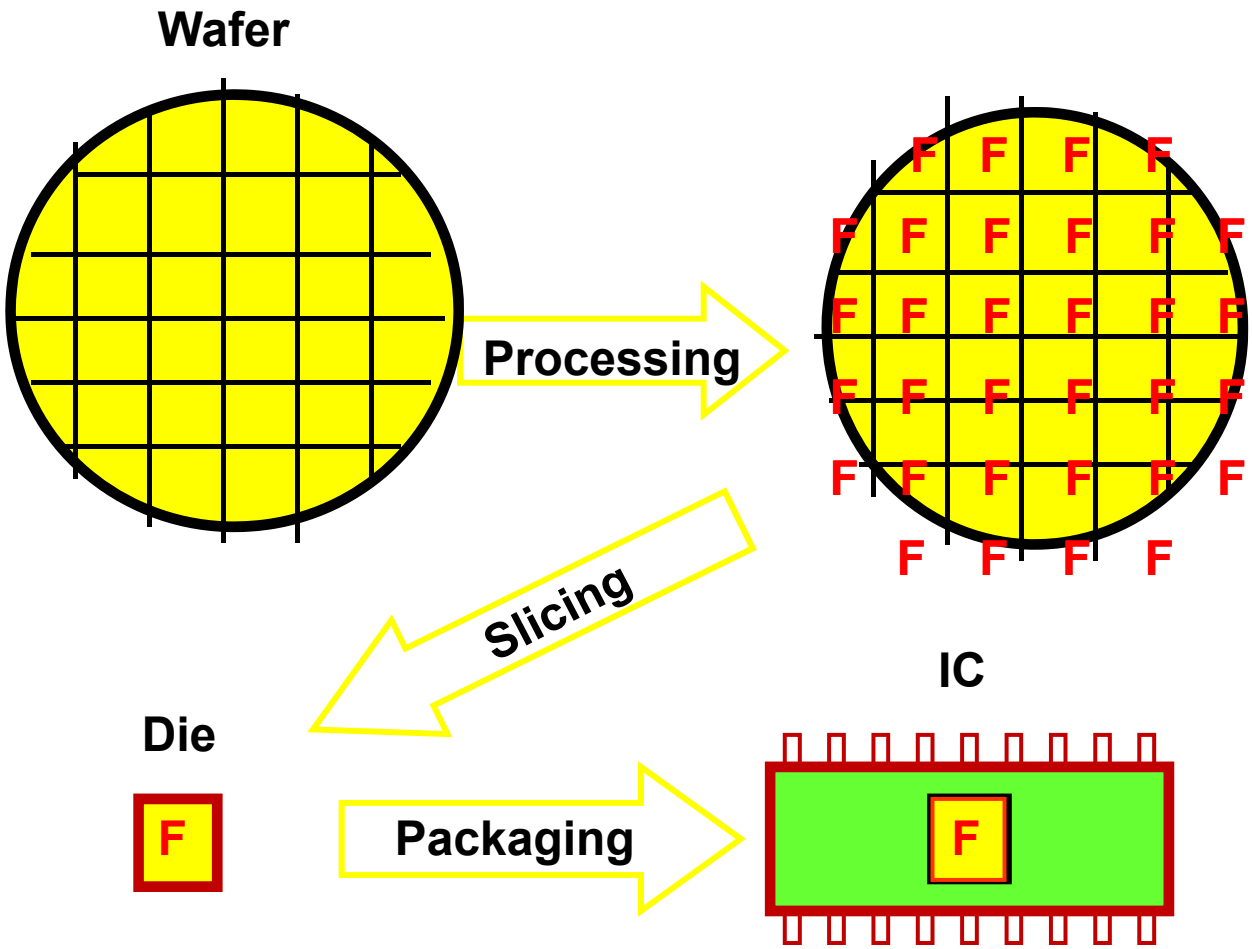
- Each copy of the integrated circuit appears in a ***die***
- Multiple dies are placed on each ***wafer***
- After fabrication, the individual dies are *separated, tested, and packaged* 切割、测试、封装。

Wafer

这个圆形叫做晶圆，每一小片被称为die，die封装后就成为芯片。

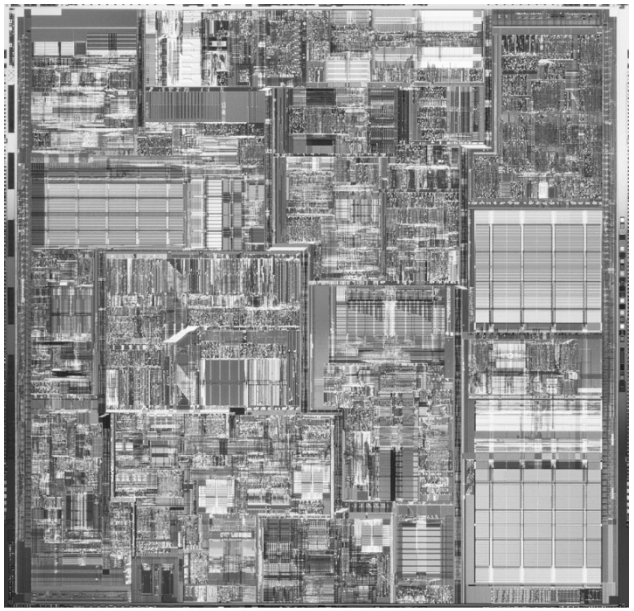


Wafer, Die, IC



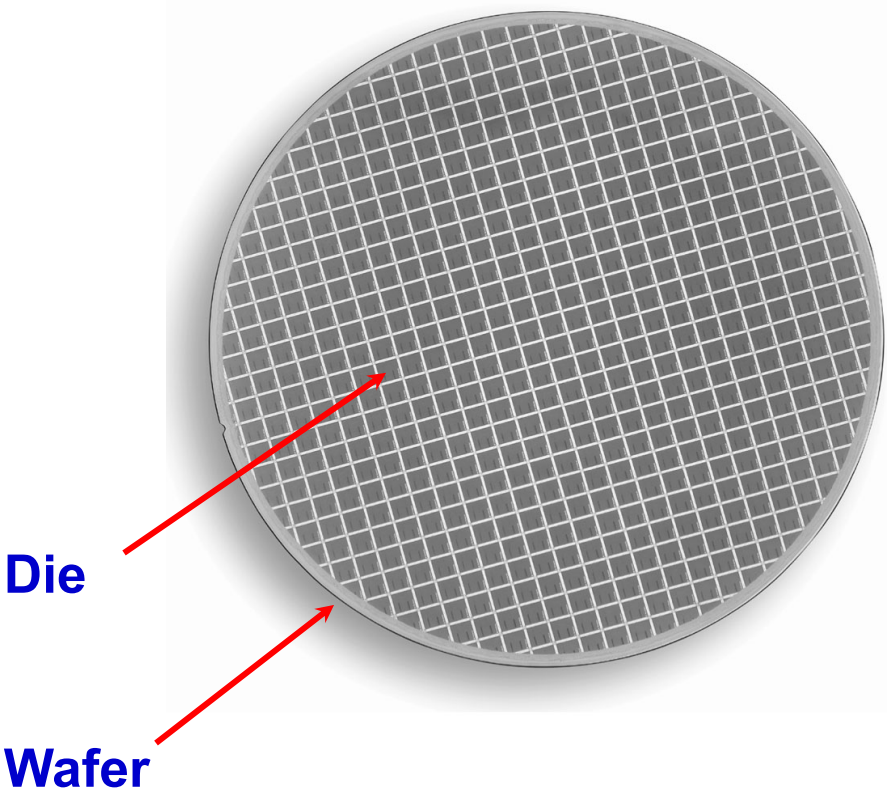
Example of Pentium 4

周围的一些边角料自然会被淘汰，这就是成品率下降原因之一。



© 2003 Elsevier Science (USA). All rights reserved.

Pentium 4 Processor



© 2003 Elsevier Science (USA). All rights reserved.

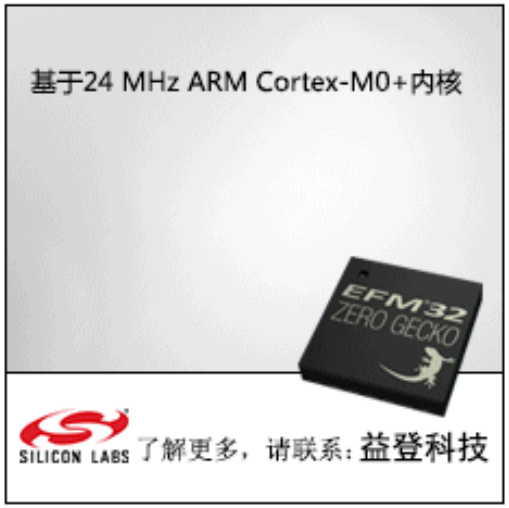
Typical Size of Industrial Wafers

首页 > 新闻 > 正文

全球首座18寸晶圆厂12月就绪

关键词： 18寸晶圆厂，半导体，台积电，晶圆

时间：2012-09-11 07:03:45　来源：苹果日报

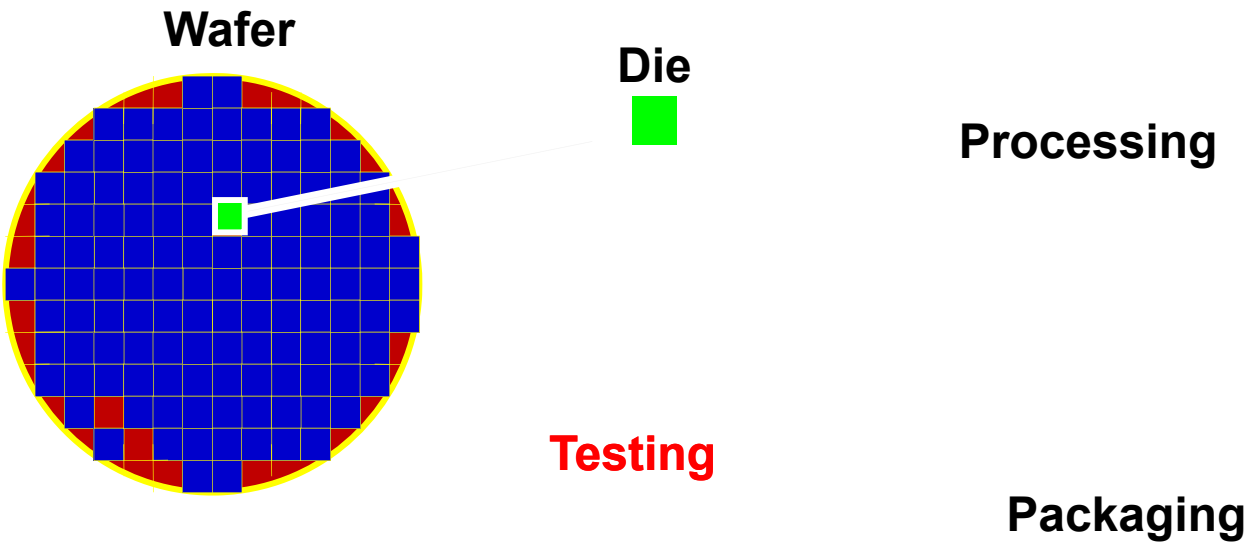


2012年国际半导体展闭幕，450mm（18寸）供应链论坛邀请到台积电、全球450mm联盟、应用材料、KLA-Tencor、LamResearch等深度探讨450mm未来发展蓝图，并率先预告世界第1座450mm晶圆厂将于今年12月准备就绪。

台积电派往全球450mm联盟的技术中心处长林进祥特别回台参加这次的论坛，他指出，世界第1座450mm晶圆厂将于今年12月准备就绪，而全球450mm联盟希望在2015年~2016年间建立18寸晶圆的测试生产线，可望陆续开始生产品质较好的生产晶圆。

Integrated Circuit Costs

$$\text{IC Cost} = \frac{\text{Die Cost} + \text{Testing cost} + \text{Packaging Cost}}{\text{Final Test Yield}}$$



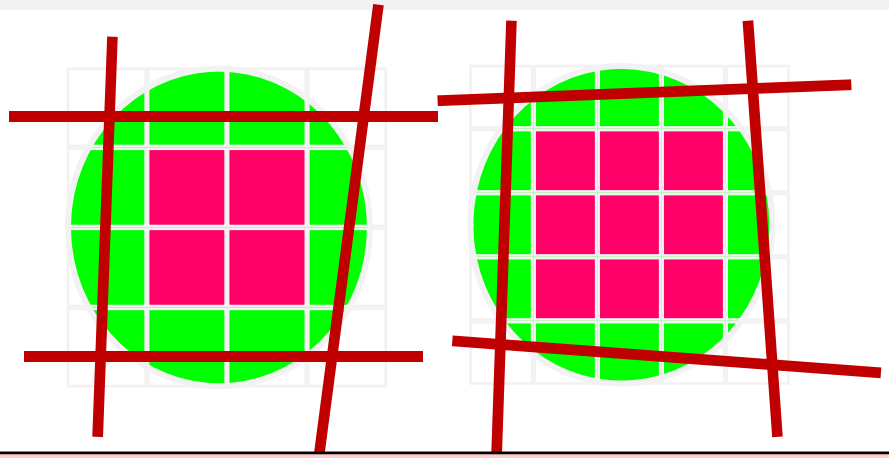
$$\text{Yields} = \frac{\text{good dies}}{\text{processed dies}}$$

Testing

Integrated Circuits Costs

$$\text{IC cost} = \frac{\text{Die cost} + \text{Testing cost} + \text{Packaging cost}}{\text{Final test yield}}$$

$$\text{Die cost} = \frac{\text{Wafer cost}}{\text{Dies per Wafer} \times \text{Die yield}}$$



$$\text{Dies per Wafer} = \frac{\pi\,(\text{Wafer_diameter}/2)^2}{\text{Die Area}} - \frac{\pi\,(\text{Wafer_diameter})}{(2 * \text{Die Area})^{\frac{1}{2}}}$$

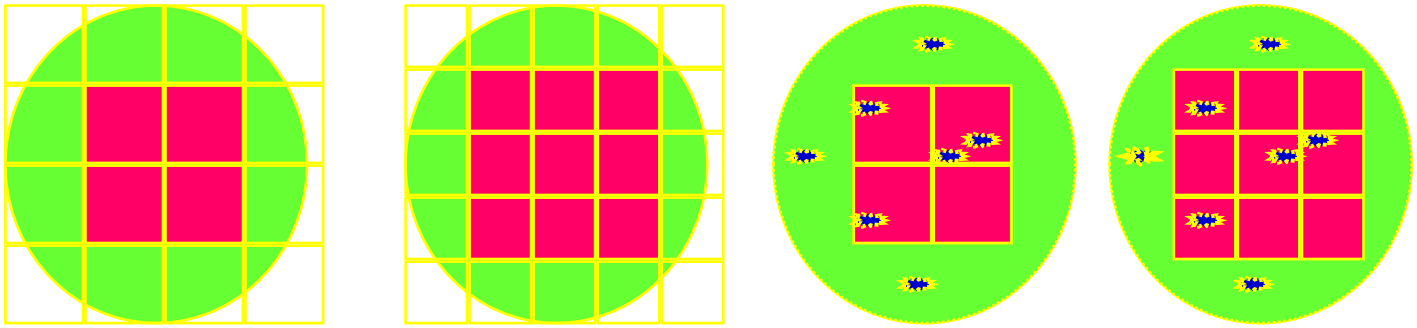
Example

- Find the number of dies per 20-cm wafer for a die that is 1.0 cm on a side and a die that is 1.5 cm on a side
- Answer

$$\text{Dies per Wafer} = \frac{\pi\,(\text{Wafer_diameter}/2)^2}{\text{Die Area}} - \frac{\pi\,(\text{Wafer_diameter})}{(2 * \text{Die Area})^{\frac{1}{2}}}$$

Integrated Circuit Cost

边缘dies与中间dies的比例、出现错误后剩余好die的比例都在证明：
更小、更多的dies会显著增加成品率、降低成本。



$$\text{Die yield} = \text{Wafer yield} \times 1/(1 + \text{Defects per unit area} \times \text{Die area})^N$$

Where N is a parameter inversely proportional to the number of mask Levels, which is a measure of the manufacturing complexity.

For today's CMOS process, good estimate is N = 11.5-15.5

Other Costs

$$\text{Die Test Cost} = \frac{\text{Test equipment Cost} * \text{Ave. Test Time}}{\text{Die Yield}}$$

Packaging Cost: depends on pins, heat dissipation, beauty, ...

Chip	Die cost	Package			Test & assembly cost	Total
		pins	type	cost		
486DX2	$12	168	PGA	$11	$12	$35
Power PC 601	$53	304	QFP	$3	$21	$77
HP PA 7100	$73	504	PGA	$35	$16	$124
DEC Alpha	$149	431	PGA	$30	$23	$202
Super SPARC	$272	293	PGA	$20	$34	$326
Pentium	$417	273	PGA	$19	$37	$473

QFP: Quad Flat Package
PGA: Pin Grid Array
BGA: Ball Grid Array

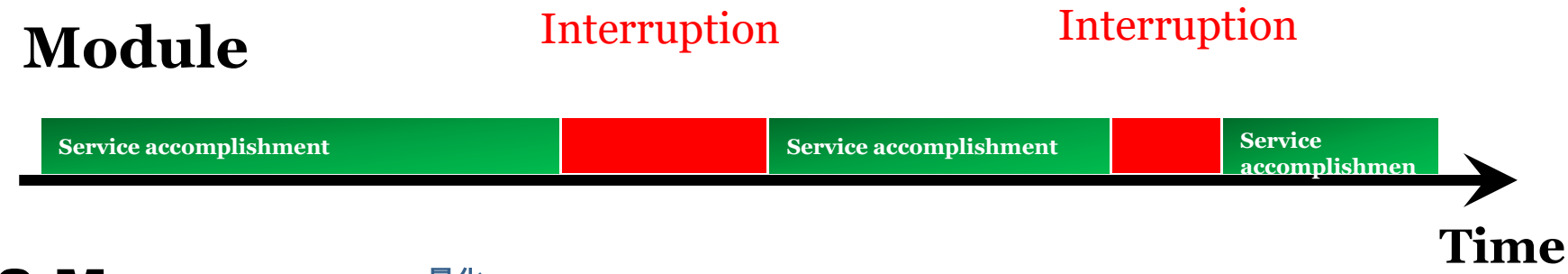
Agenda

- Introduction
- Classes of Computers
- Defining Computer Architecture
- Trends in Technology
- Trends in Power and Energy in ICs
- Trends in Cost
- 1.7 Dependability
- Measuring Performance
- Quantitative Principles

Dependability

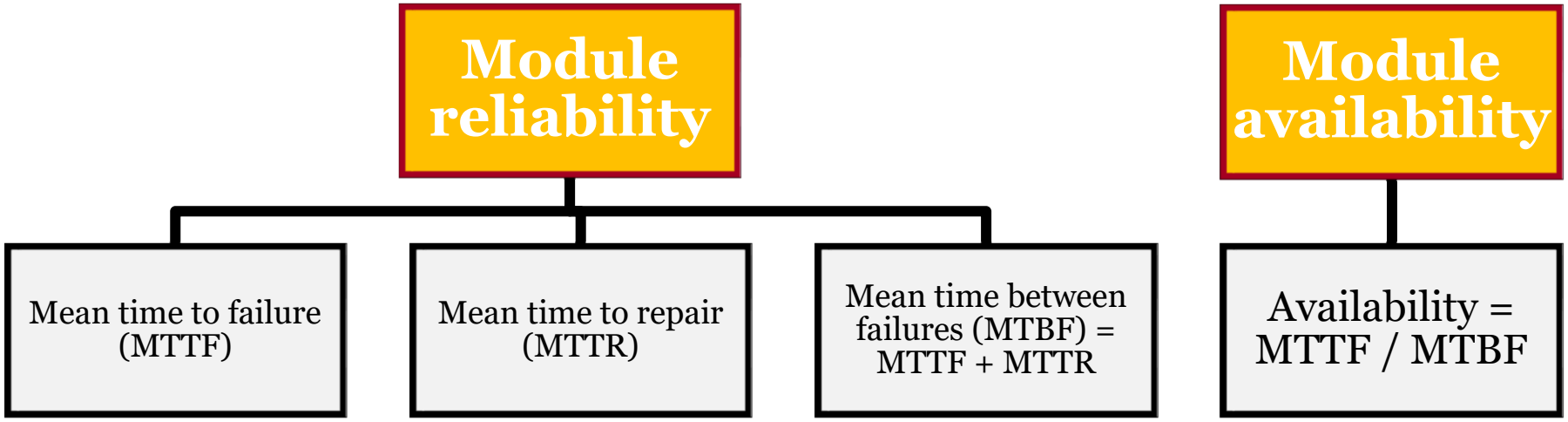
可靠性：
我们总是希望计算机是可靠的
然而怎么定义、比较可靠性呢？

Module



2 Measures:

量化



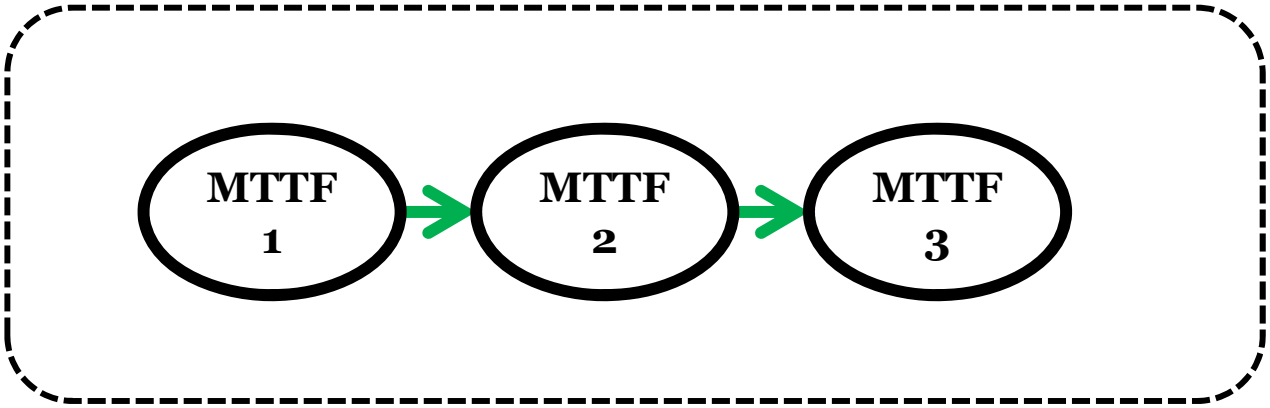
TTF：time to failure
MTTF：期望TTF。我们希望mttf尽可能更大。

TTR：time to repair

MTBF：ttf+ttr

占比，希望中断与错误的处理很快就能完成。

MTTF of a System

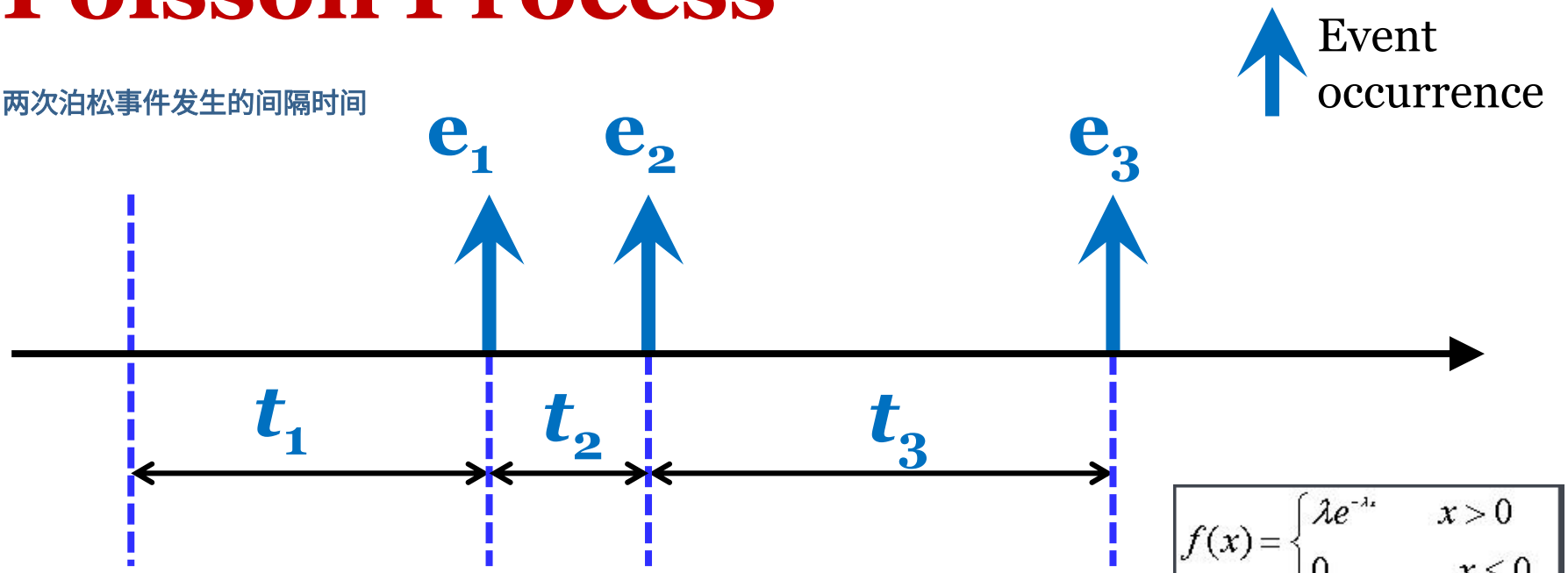


System

- The MTTF of each module is *exponentially distributed* (independent of age)
- Modules are independent of each other
- What is the MTTF of the system?

Exponential Distribution and Poisson Process

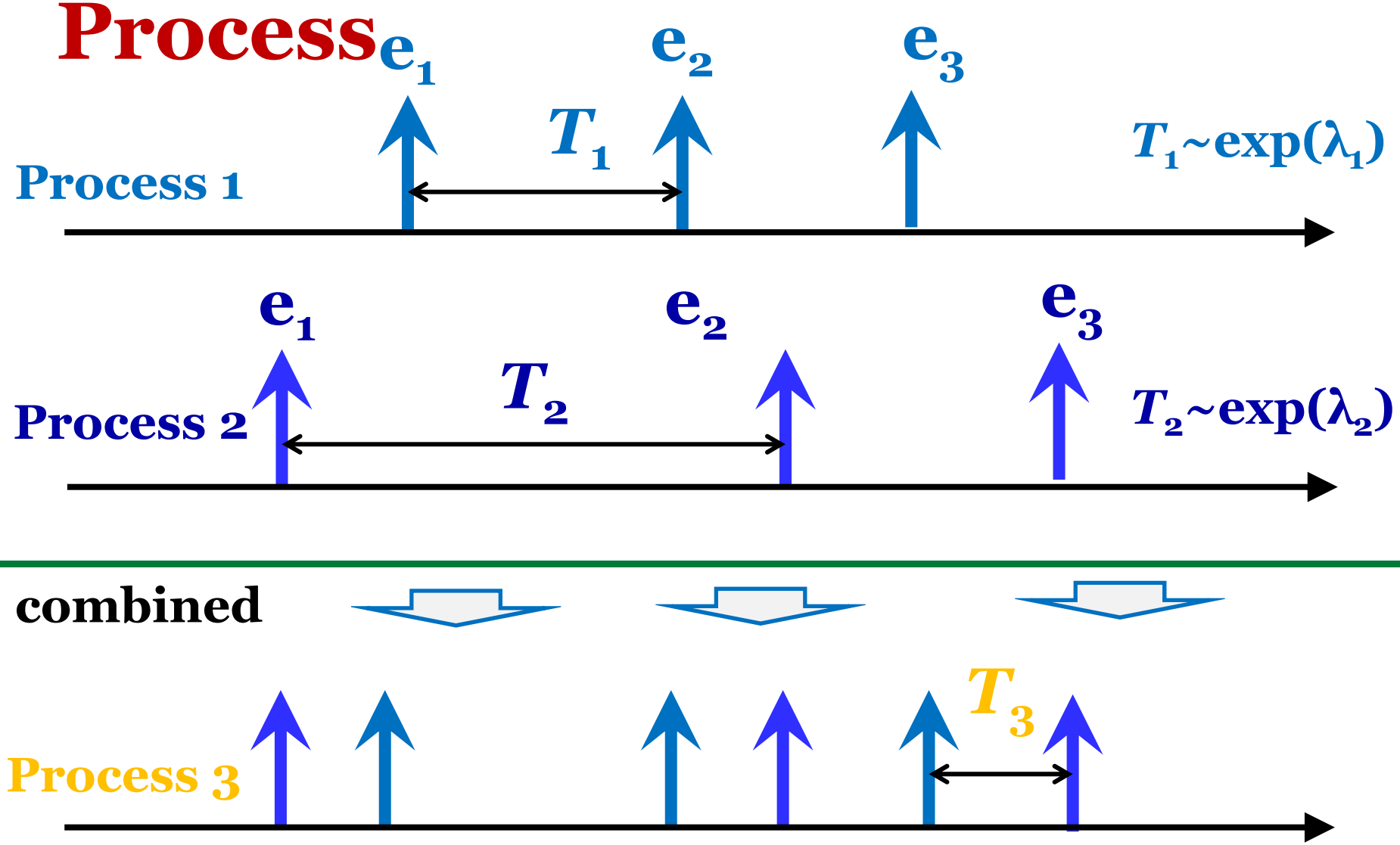
指数分布：两次泊松事件发生的间隔时间



$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x > 0 \\ 0 & x \le 0 \end{cases}$$

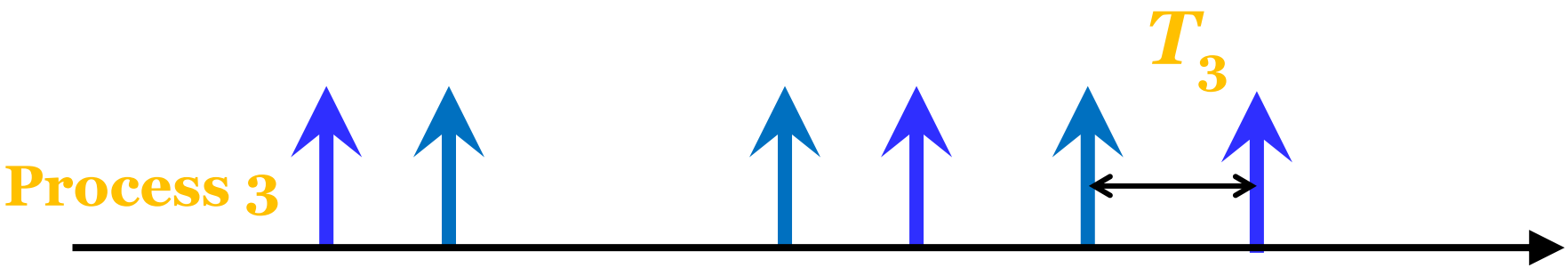
- The time, *T*, between two events is a R.V. following the *exponential distribution*
- The arrivals of events follow the *Poisson Process*. Example, incoming calls of a hotline service

Characteristics of Poisson Process



Characteristics of Poisson Process

若两个子结构的T服从指数分布，那么整个计算机也服从指数分布，并且参数相加。

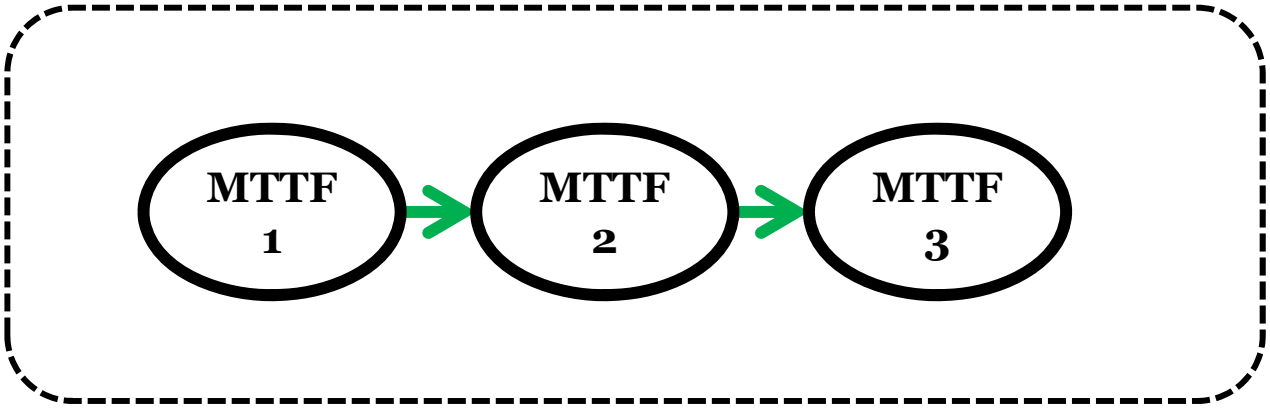


What process? T_3 follows what distribution?

$$T_3 \sim \exp(\lambda_3)$$

$$\lambda 3 = \lambda 1 + \lambda 2$$

MTTF of a System



System

What is the MTTF of the system?

$$MTTF_{system} = \frac{1}{\frac{1}{MTTF_1} + \frac{1}{MTTF_2} + \frac{1}{MTTF_3}}$$

Exercise

Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Assumptions:

MTTF of the whole System?

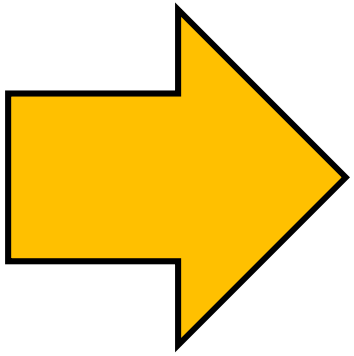
- Exponential distributed
- Failures are independent

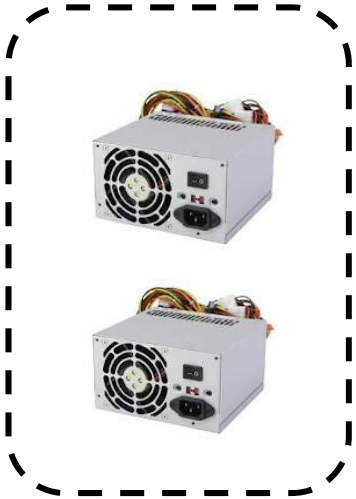
How to Increase Dependability?

Redundancy!

- In time (repeat the operation to see if it is still erroneous)
- In resources (duplicate resources)

用容错换来更大的可靠性。



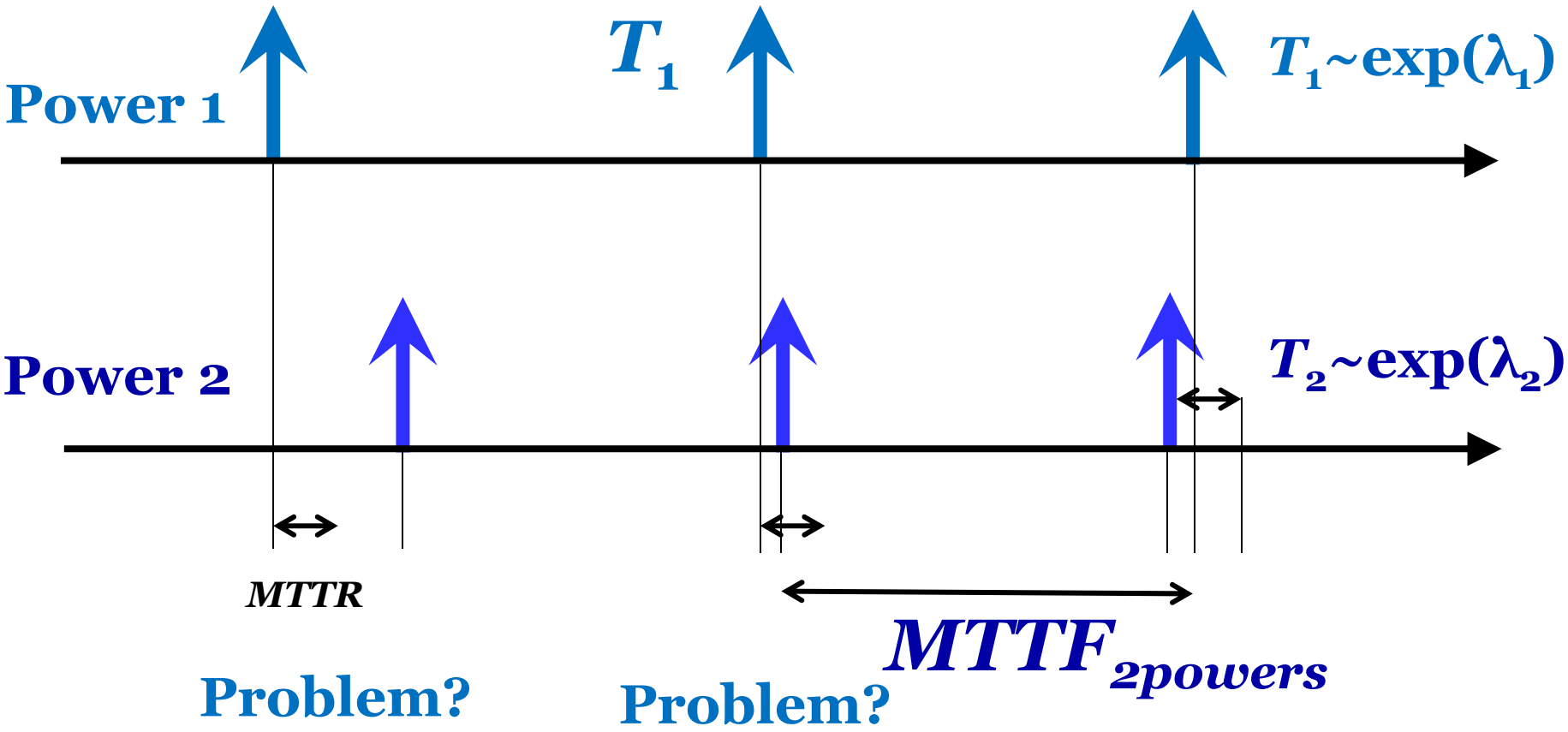


200,000-hour MTTF

10-hour MTTR

MTTF of two power supplies?

Computing the MTTF of Two Suppliers



$$MTTF_{2powers} = \frac{\frac{MTTF}{2}}{\frac{MTTR}{MTTF}}$$

同时off

Agenda

- Introduction
- Classes of Computers
- Defining Computer Architecture
- Trends in Technology
- Trends in Power and Energy in ICs
- Trends in Cost
- Dependability
- 1.8 Measuring Performance
- Quantitative Principles

Typical Performance Metrics

Response time (or execution time)

- The time between the start and the completion of an event

Throughput

- The total amount of work done in a unit time

Benchmarks

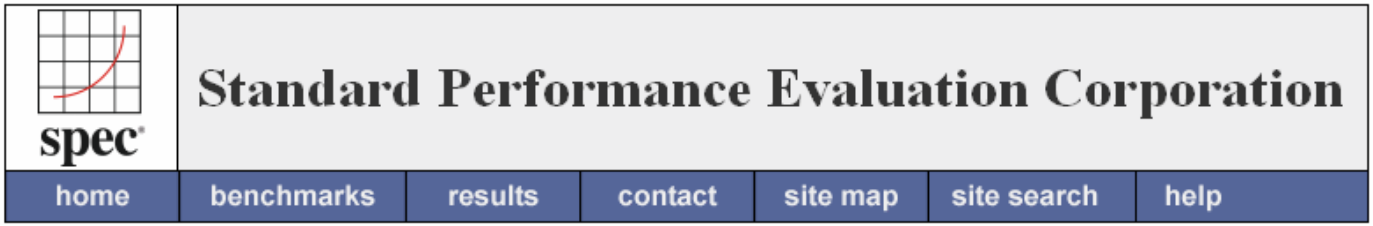
Benchmark: a common program for testing the execution times of computers

Programs lead to poor performance indication

- Kernels (e.g. matrix multiply)
- Toy programs (e.g. sorting)
- Synthetic benchmarks (e.g. Dhrystone)

Benchmark suit: collection of benchmark programs

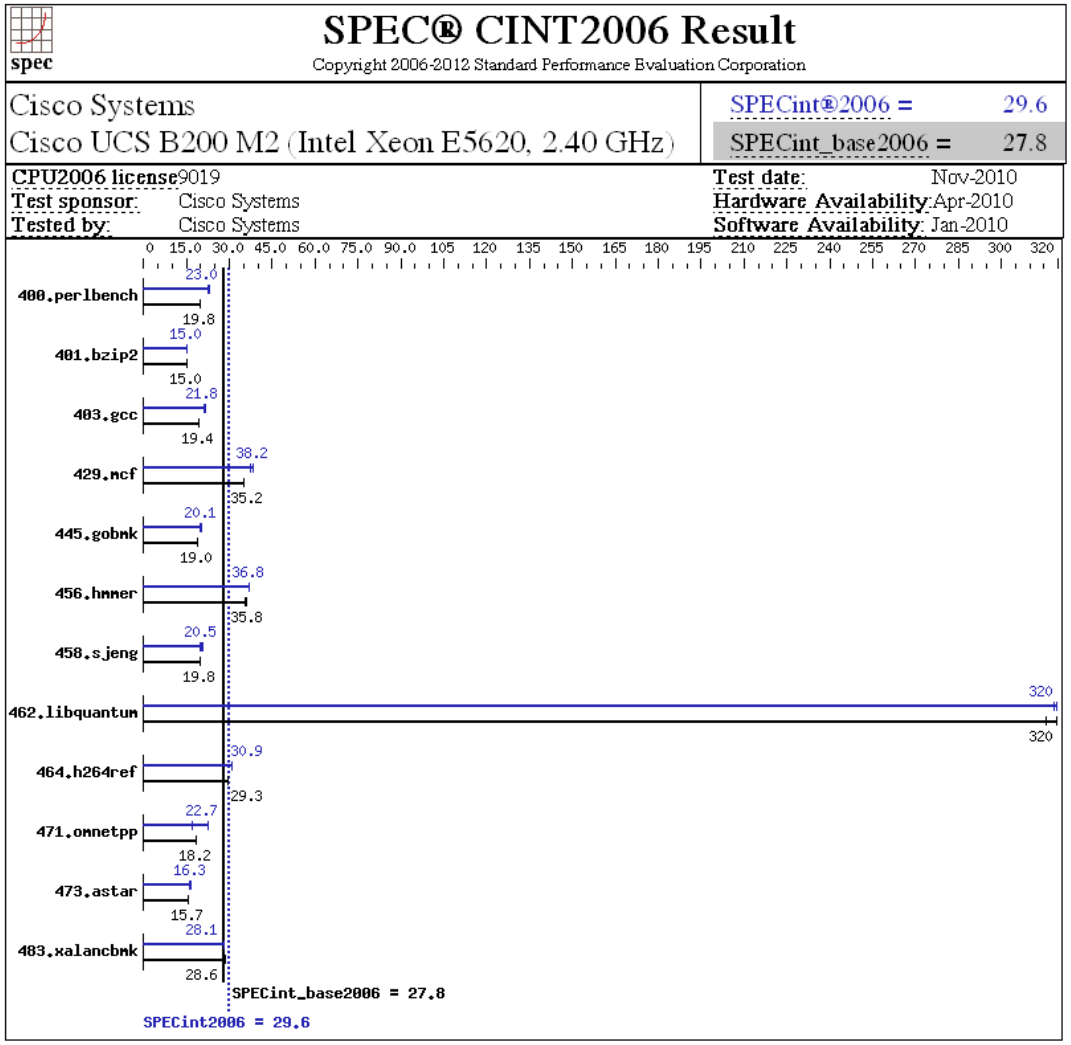
SPEC



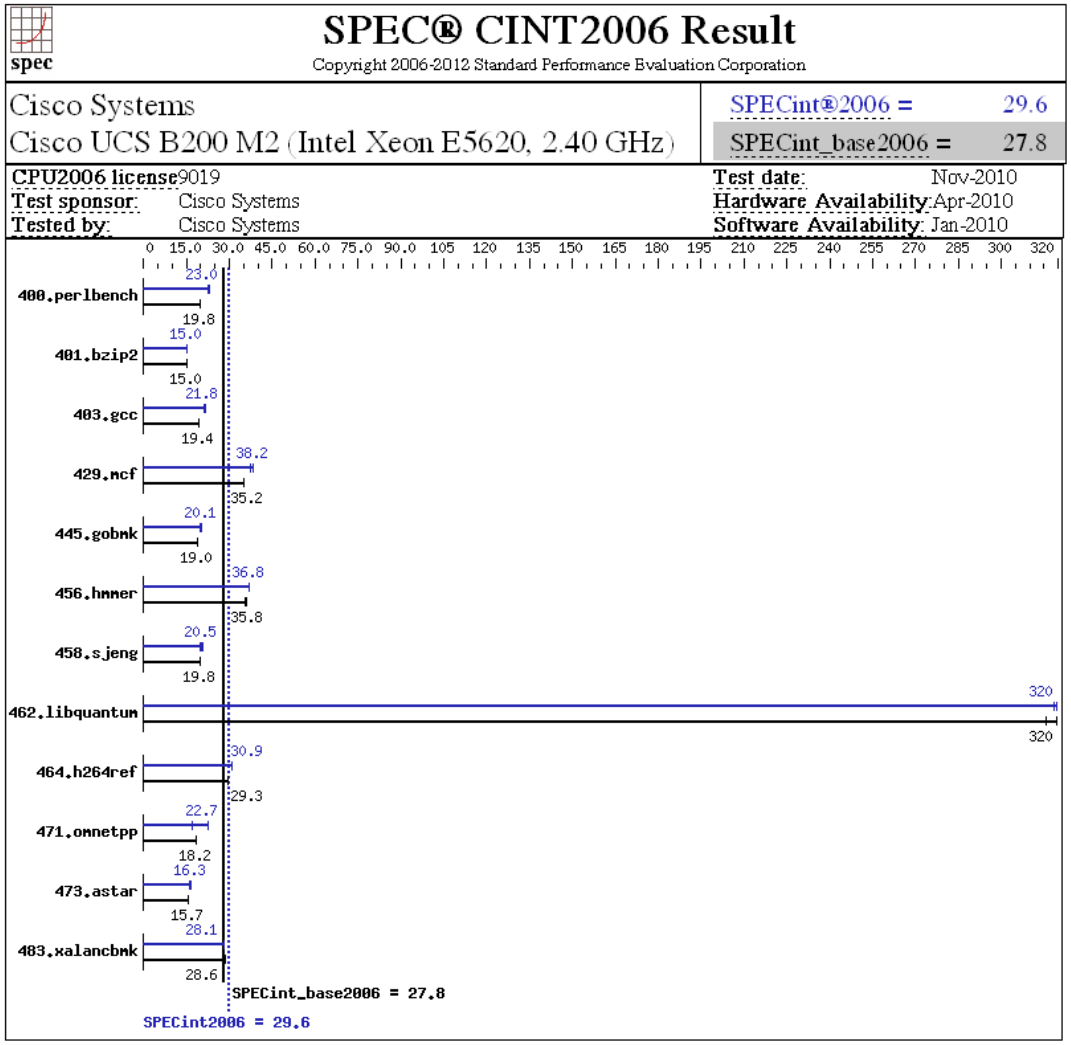
SPEC: Standard Performance Evaluation Corporation

- *A non-profit corporation* formed to establish, maintain and endorse a standardized set of relevant benchmarks that can be applied to the *newest generation of high-performance computers*.
- SPEC develops *benchmark suites* and also reviews and publishes *submitted results* from member organizations and other benchmark licensees.

Example SPEC Result Report



Example SPEC Result Report



400.perlbench
SPEC CPU2006 Benchmark Description

Benchmark Name

400.perlbench

Benchmark Authors

Larry Wall, et. al.

Benchmark Program General Category

Programming language

Benchmark Description

400.perlbench is a cut-down version of Perl v5.8.7, the popular scripting language. SPEC's version of Perl has had most of OS-specific features removed. In addition to the core Perl interpreter, several third-party modules are used:

- SpamAssassin v2.61
- Digest-MD5 v2.33
- HTML-Parser v3.35
- MHonArc v2.6.8
- IO-stringy v1.205
- MailTools v1.60
- TimeDate v1.16

Sources for all of the freely-available components used in 400.perlbench can be found in $SPEC/redistributable_sources/original/400.perlbench/ on your SPEC CPU2006 DVD.

Reporting Performance Results

How do we summarize performance, given the execution times of a set of benchmarks?

- Example as shown on the right.

Benchmarks	Opteron time (sec)	Itanium 2 time (sec)
wupwise	51.5	56.1
swim	125.0	70.7
mgrid	98.0	65.8
applu	94.0	50.9
mesa	64.6	108.0
galgel	86.4	40.0
art	92.4	21.0
equake	72.6	36.3
facerec	73.6	86.9
ammp	136.0	132.0
lucas	88.8	107.0
fma3d	120.0	131.0
sixtrack	123.0	68.8
apsi	150.0	231.0

Option 1: Arithmetic Mean

- The arithmetic mean of $x_1, x_2, \ldots, x_n$ is

$$Mean_{arith} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- The ***problem*** of using arithmetic mean?
  - Benchmark programs with longer execution times would become more important
  - Example: 4 numbers **(5, 6, 5, 7, 100)**

Option 2: Weighted Arithmetic Mean

- To add a ***weighting factor*** to each benchmark program
- The *question*: how do we set weight factors?

- ***Possible solution***: use weights to make programs execute an equal time on a reference computer.
- ***Problem***: the reference computer would become crucial

SPEC Approach: Performance Ratio

- Instead of using absolute execution times, use ***the ratio*** of performance to *a reference computer*
- **A good property of using performance ratio:** the selection of reference computer is **irrelevant**

$$1.25 = \frac{\text{SPECRatio}_A}{\text{SPECRatio}_B} = \frac{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_A}}{\dfrac{\text{Execution time}_{\text{reference}}}{\text{Execution time}_B}} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = \frac{\text{Performance}_A}{\text{Performance}_B}$$

SPEC Approach: Geometric Mean (Cont)

- The ***geometric mean*** of $x_1, x_2, \ldots, x_n$ is

$$Mean_{Geometric} = \sqrt[n]{\prod_{i=1}^{n} x_i}$$

- Two **good properties** of geometric mean
  - 1) The geometric mean of the ratios = the ratio of geometric means
  - 2) The choice of the reference computer is irrelevant

Agenda

- Introduction
- Classes of Computers
- Defining Computer Architecture
- Trends in Technology
- Trends in Power and Energy in ICs
- Trends in Cost
- Dependability
- Measuring Performance
- **1.9 Quantitative Principles**

定量的原则

Principles for Computer Design

Take advantage of parallelism 使用并行的优势

- Data level parallelism and task level parallelism 数据并发与任务并发
- Pipelining, set-associative caches 例如流水线、缓存
- Multicore, multiprocessor, vector 多处理器、多核、向量

Principle of locality 局部性原理

引出分级存储概念

- Program intends to reuse data and instructions they have used recently
- Temporal locality and spatial locality

时间与空间性的"locality"
- 若访问了一条数据、指令，则会未来继续高频访问
- 若访问了一条数据，则在未来会访问它空间附近的其他指令。

Focus on common case 关注经常性事件

- Favor the frequent case over the infrequent case

Amdahl's Law

The law defines the speedup by using a particular feature

加速经常性事件往往有效，但是对整体的加速比却受到其他未加速部分的限制

$$\text{Execution time}_{\text{new}} = \text{Execution time}_{\text{old}} \times \left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)$$

$$\text{Speedup}_{\text{overall}} = \frac{\text{Execution time}_{\text{old}}}{\text{Execution time}_{\text{new}}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

Amdahl's law states that the performance improvement of using a new feature is limited by the fraction of the time the new feature can be used.

阿姆达尔定律指出，使用新功能的性能提升受限于新功能可用时间的百分比。
解决问题时更注重经常性事件

Processor Performance

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

DEFINITION: $$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

Three Factors for Processor Improvement

Clock cycle time	Hardware technology and organization
CPI	Organization and instruction set architecture
Instruction count	Instruction set architecture and compiler technology

Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i$$

$$\text{CPU time} = \left(\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i \right) \times \text{Clock cycle time}$$

$$\text{CPI} = \frac{\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i}{\text{Instruction count}} = \sum_{i=1}^{n} \frac{\text{IC}_i}{\text{Instruction count}} \times \text{CPI}_i$$